

Excellent. This is exactly the kind of detail that makes implementation smooth.

Since your project already follows the architecture:

```
Route
  ↓
Controller
  ↓
Service
  ↓
Repository
```

the API contracts should be **implementation independent**. A frontend developer should be able to start development without reading the backend code.

Environment Support Module API Contracts

1. Get All Environments

Purpose

Returns all environments of a project.

Endpoint

```
http
```

```
GET /api/v1/projects/:projectId/environments
```

Authorization

```
Owner
Admin
Developer
Viewer
```

Request

No Body

Example

```
GET /api/v1/projects/64ab23/environments
```

Success Response

JSON

```
{
  "success": true,
  "message": "Environments fetched successfully",
  "data": [
    {
      "id": "env1",
      "name": "Development",
      "slug": "development",
      "color": "#22c55e",
      "order": 1,
      "isDefault": true
    },
    {
      "id": "env2",
      "name": "Staging",
      "slug": "staging",
      "color": "#f59e0b",
      "order": 2,
      "isDefault": true
    },
    {
      "id": "env3",
      "name": "Production",
      "slug": "production",
      "color": "#ef4444",
      "order": 3,
      "isDefault": true
    }
  ]
}
```

Errors

Code	Reason
401	Unauthorized
403	Not a project member
404	Project not found

2. Create Environment

Purpose

Creates a custom environment inside a project.

Endpoint

```
http
POST /api/v1/projects/:projectId/environments
```

Authorization

```
Owner
Admin
```

Request

```
JSON
{
  "name": "QA",
  "description": "Testing Environment",
  "color": "#3b82f6"
}
```

Validation

Field	Rule
name	required
name	3-30 chars
name	unique inside project
description	optional
color	optional

Success

```
JSON
{
  "success":true,
  "message":"Environment created successfully",
  "data":{
    "id":"env5",
    "name":"QA",
    "slug":"qa",
    "color":"#3b82f6",
    "isDefault":false
  }
}
```

Backend Side Effect

This endpoint must also

```
Fetch all Features
↓
Create FeatureConfiguration
for every feature
```

This is important.

Errors

Code	Reason
400	Validation Failed
401	Unauthorized
403	Permission Denied
404	Project Not Found
409	Environment Already Exists

3. Update Environment

Endpoint

http

PATCH /api/v1/environments/:environmentId

Authorization

Owner

Admin

Request

JSON

```
{
  "name": "Testing",
  "color": "#2563eb",
  "order": 4
}
```

Validation

Cannot change

projectId

isDefault

Response

JSON

```
{
  "success":true,
  "message":"Environment updated successfully",
  "data":{
    ...
  }
}
```

Errors

404

Environment Not Found

409

Duplicate Name

4. Delete Environment

Only custom environments.

Endpoint

http

DELETE /api/v1/environments/:environmentId

Authorization

Owner

Flow

Find Environment

↓

Check isDefault

↓

If true

Reject

↓

Delete FeatureConfigurations

↓

Delete Environment

Success

JSON

```
{
  "success":true,
  "message":"Environment deleted successfully"
}
```

Errors

400

Cannot delete default environment

5. Get Feature Configurations

This endpoint loads all environment configurations for one feature.

Endpoint

http

GET /api/v1/features/:featureId/configurations

Response

JSON

```
{
  "success": true,
  "data": [
    {
      "environmentId": "env1",
      "environmentName": "Development",
      "enabled": true,
      "rolloutPercentage": 100,
      "killSwitch": false
    },
    {
      "environmentId": "env2",
      "environmentName": "Staging",
      "enabled": true,
      "rolloutPercentage": 50,
      "killSwitch": false
    },
    {
      "environmentId": "env3",
      "environmentName": "Production",
      "enabled": true,
      "rolloutPercentage": 5,
      "killSwitch": false
    }
  ]
}
```

```
}  
]  
}
```

6. Update Feature Configuration

Probably the most important endpoint.

Endpoint

http

PATCH /api/v1/features/:featureId/configurations/:environmentId

Request

JSON

```
{  
  "enabled":true,  
  "rolloutPercentage":25,  
  "killSwitch":false  
}
```

Validation

Rollout

$0 \leq \text{rollout} \leq 100$

Enabled

Boolean

Kill Switch

Boolean

Response

```
JSON

{
  "success":true,
  "message":"Configuration updated successfully",
  "data":{
    ...
  }
}
```

7. Reorder Environments (Optional but Recommended)

Instead of updating one by one.

Endpoint

http

PATCH /api/v1/projects/:projectId/environments/reorder

Request

```
JSON

{
  "environmentIds":[
    "env2",
    "env1",
    "env3"
  ]
}
```

Backend

Loop

↓

Update order

↓

Return Success

Standard Response Format

Every endpoint should follow the same response structure.

Success

JSON

```
{  
  "success": true,  
  "message": "...",  
  "data": {}  
}
```

Error

JSON

```
{  
  "success": false,  
  "message": "Environment already exists",  
  "errors": [  
    {  
      "field": "name",  
      "message": "Duplicate environment"  
    }  
  ]  
}
```

API Implementation Notes (for Backend Developer)

Transactions

These operations **must** use MongoDB transactions because they affect multiple collections:

- **Create Project** → Project + 3 default Environments.
- **Create Feature** → FeatureFlag + FeatureConfiguration for every Environment.
- **Create Environment** → Environment + FeatureConfiguration for every existing Feature.
- **Delete Environment** → Delete Environment + all related FeatureConfiguration documents.

Without transactions, a partial failure could leave the database in an inconsistent state (for example, an Environment exists but its configurations do not).

Final Review

As the architect, I'd approve these contracts with **one small improvement** to keep the API RESTful and future-proof.

Instead of:

```
GET /features/:featureId/configurations
PATCH /features/:featureId/configurations/:environmentId
```

I recommend treating **FeatureConfiguration** as its own resource:

```
GET /features/:featureId/configurations
GET /feature-configurations/:id
PATCH /feature-configurations/:id
```

This has a few advantages:

- Every configuration has its own identity (`id`).
- Future modules (Audit Log, SDK, Analytics) can reference a configuration directly.
- URLs stay cleaner if more configuration fields are added later.

This is a small architectural improvement, but it aligns better with the way the data model is designed.